

Technical Software Requirements Specification (SRS)

Rwanda NBSAP Monitoring Dashboard 2025–2030

Prepared for: Rwanda Environment Management Authority (REMA) **Document Reference:** NBSAP-SRS-2026-004 **Version:** 1.0 **Date:** June 2026 **Classification:** Official — Government Use

Document Control

Field	Value
Document Title	Technical Software Requirements Specification
Project	Rwanda NBSAP Monitoring Dashboard
Prepared by	NBSAP Dashboard Development Team
Approved by	REMA — Department of Biodiversity & Landscape Management
Status	Final

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the technical architecture, interfaces, data structures, and implementation details of the Rwanda NBSAP Monitoring Dashboard. It serves as the definitive technical reference for development, maintenance, and audit purposes.

1.2 System Overview

The NBSAP Monitoring Dashboard is a single-page application (SPA) built on a serverless Backend-as-a-Service (BaaS) architecture. It provides real-time biodiversity monitoring, multi-stakeholder reporting, automated metric computation, and geographic data visualisation.

Attribute	Value
System Name	Rwanda NBSAP Monitoring Dashboard

Live URL	https://nbsap-dashboard-rw.vercel.app
Architecture	Serverless BaaS (Supabase + Vercel)
Frontend	React 18 + TypeScript + Vite
Backend	PostgreSQL 15 + Supabase Auth + Storage + Realtime
Deployment	Vercel (CDN) + Supabase Cloud
Codebase	~15,000+ lines of TypeScript across 50+ components

2. Technology Stack

2.1 Frontend Stack

Technology	Version	Purpose
React	18.2	Component-based UI framework
TypeScript	5.0	Static type checking and code safety
Vite	4.3	Build tool and development server
React Router	v6	Client-side routing with protected routes
Leaflet.js	Latest	Interactive map rendering
Recharts	Latest	Chart and data visualisation
Lucide React	Latest	Icon library
React Hot Toast	Latest	User notification toasts

2.2 Backend Stack

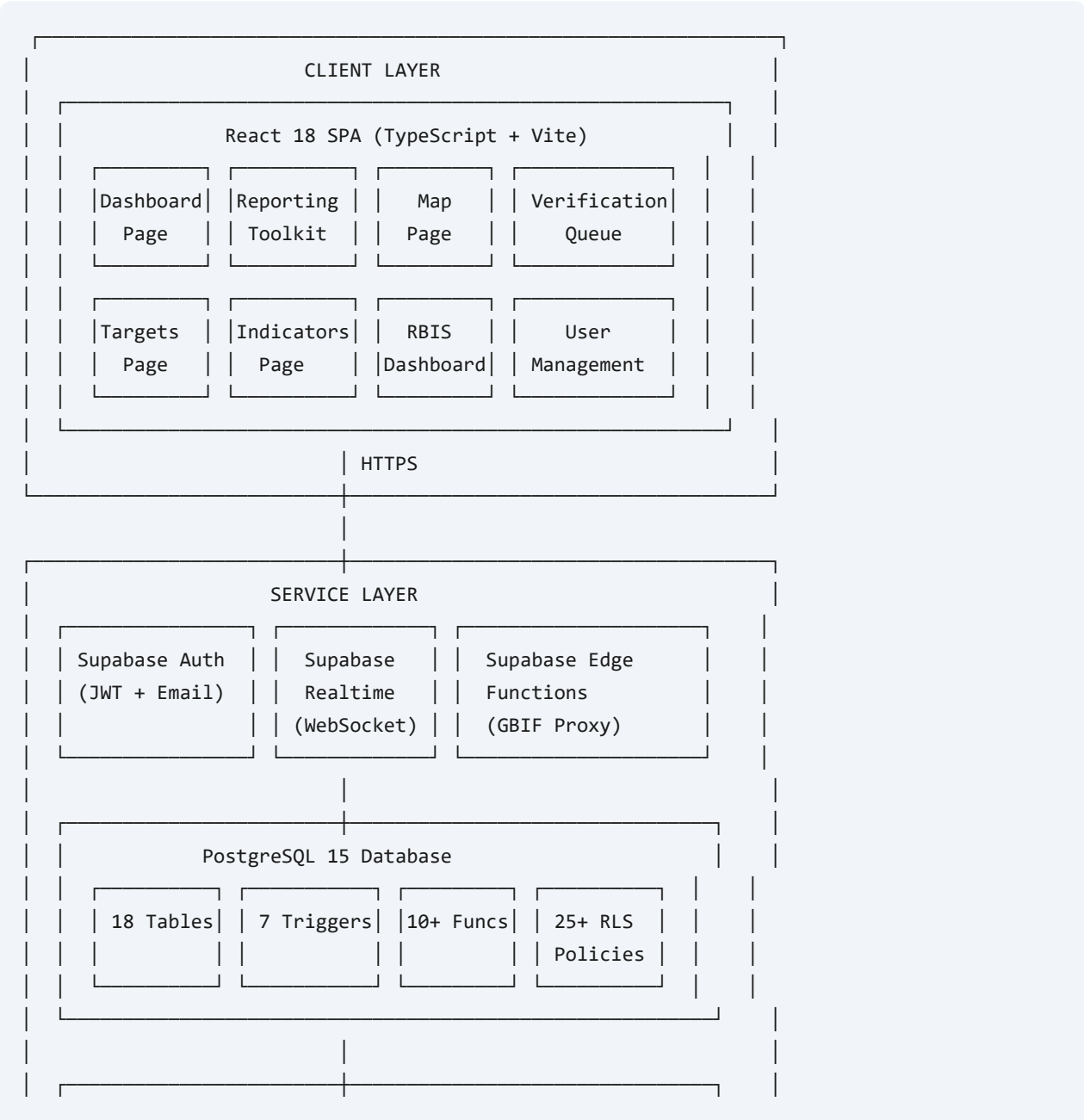
Technology	Version	Purpose
PostgreSQL	15	Relational database with JSONB support
Supabase Auth	Latest	JWT-based authentication and session management
Supabase Storage	Latest	S3-compatible file storage for report attachments
Supabase Realtime	Latest	WebSocket-based real-time data push
Supabase Edge Functions	Deno runtime	Serverless functions (GBIF API proxy)
Row-Level Security	PostgreSQL native	Fine-grained data access control

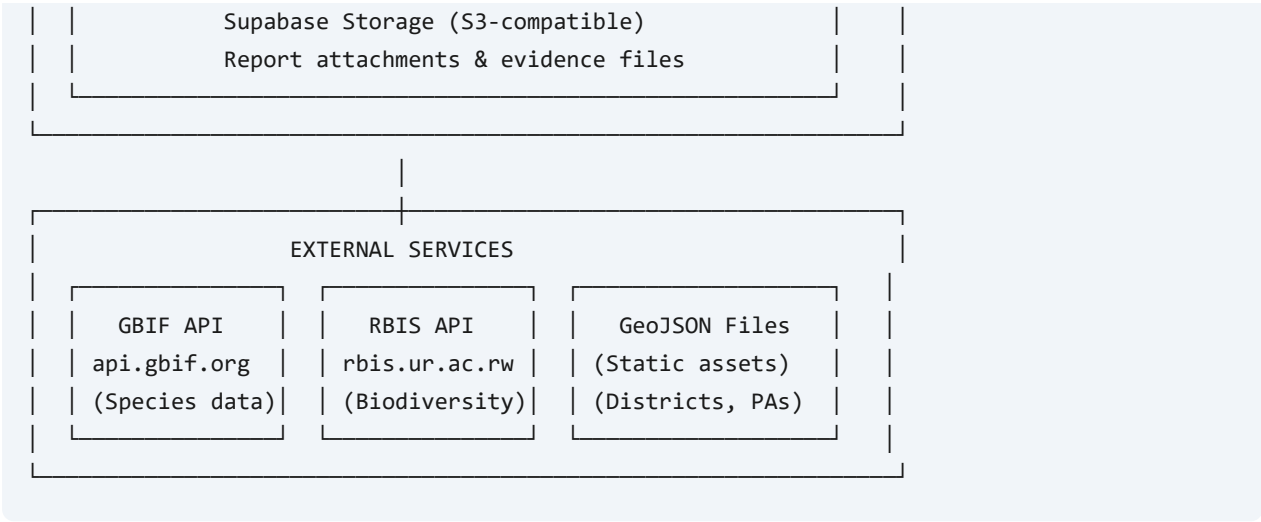
2.3 Infrastructure

Component	Provider	Purpose
Frontend Hosting	Vercel	Global CDN with auto-deploy on git push
Backend Hosting	Supabase Cloud	Managed PostgreSQL, Auth, Storage, Realtime
DNS & SSL	Vercel	Automatic HTTPS certificate provisioning
CI/CD	Vercel	Git push → auto-build → deploy pipeline
Version Control	GitHub	Source code management

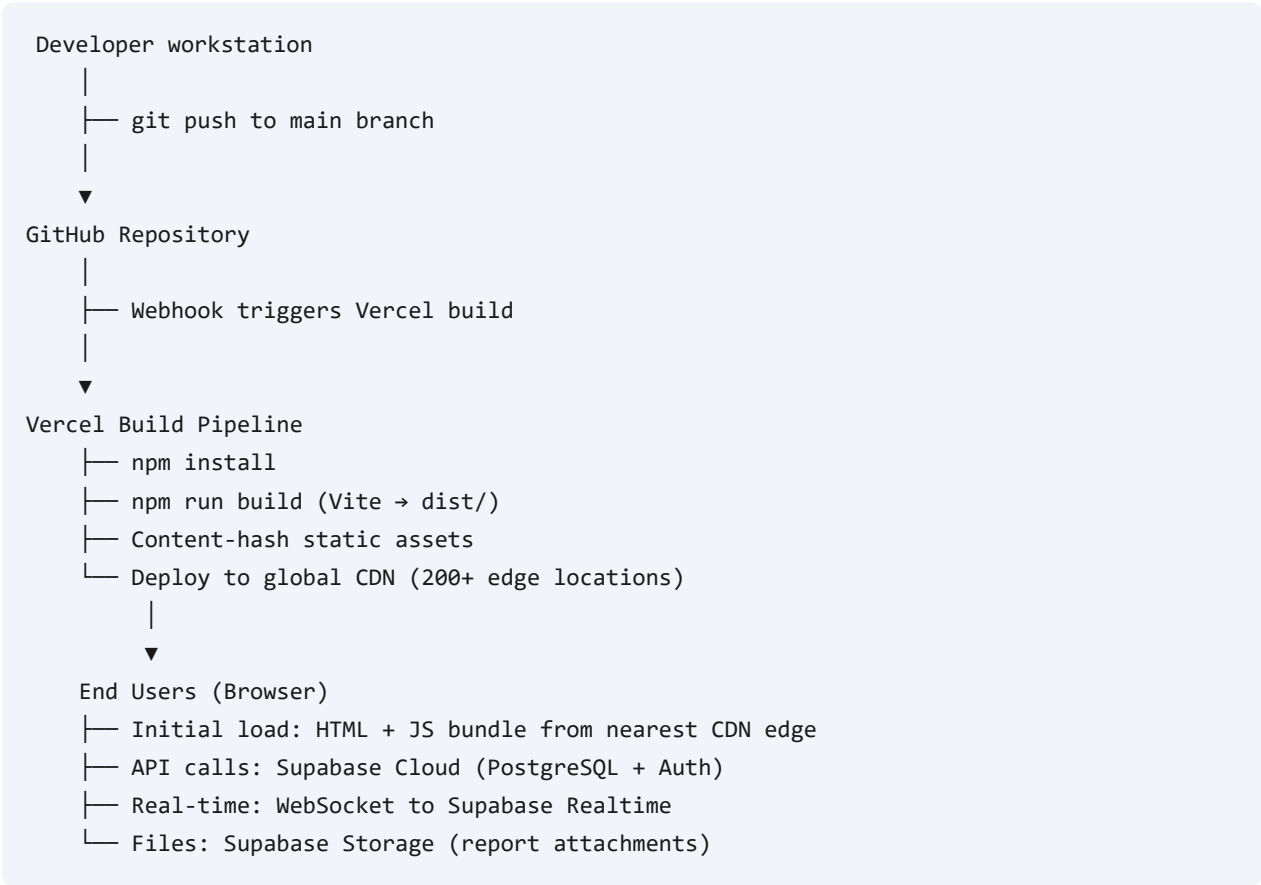
3. System Architecture

3.1 High-Level Architecture Diagram





3.2 Deployment Architecture



4. Application Module Specification

4.1 Page Inventory

Page	Route	Component	Access Level
Dashboard	/dashboard	DashboardPage.tsx	All authenticated users

National Targets	/targets	NationalTargetsPage.tsx	All authenticated users
Indicators	/indicators	IndicatorsPage.tsx	All authenticated users
Map	/map	MapPage.tsx	All authenticated users
Reporting Toolkit	/reporting	ReportingToolkitPage.tsx	Reporters + Admins
Verification Queue	/verification	VerifQueuePage.tsx	Reviewers + Admins
Reports	/reports	ReportsPage.tsx	Analytics-capable roles
RBIS Dashboard	/rbis	BiodiversityDataPage.tsx	All authenticated users
Compliance	/compliance	CompliancePage.tsx	Compliance-capable roles
Risk Register	/risk	RiskPage.tsx	Risk-capable roles
Stakeholders	/stakeholders	StakeholdersPage.tsx	Admins
Data Pipeline	/pipeline	DataPipelinePage.tsx	Admins
User Management	/users	UserManagementPage.tsx	Admins only
Role Requests	/role-requests	RoleRequestsPage.tsx	Admins only
Settings	/settings	SettingsPage.tsx	All authenticated users
Adaptive Management	/adaptive	AdaptiveManagementPage.tsx	Policy roles

4.2 Component Architecture

```

src/
├─ components/
│  ├─ common/                # Shared UI components
│  │  ├─ DatePicker.tsx
│  │  ├─ YearSelector.tsx
│  │  └─ ValidationErrors.tsx
│  ├─ map/                   # GIS components
│  │  ├─ GBIFOccurrencesOverlay.tsx
│  │  ├─ ProtectedAreasOverlay.tsx
│  │  ├─ RiverNetworkOverlay.tsx
│  │  ├─ LakesOverlay.tsx
│  │  ├─ LayerSwitcher.tsx
│  │  ├─ MapControls.tsx
│  │  ├─ MapLegend.tsx
│  │  └─ DistrictDetailPanel.tsx
│  └─ panels/                # Data display panels
│     ├─ BiodiversityIndexPanel.tsx
│     └─ GBIFLiveCounter.tsx

```

- ├── HotspotsListPanel.tsx
- ├── ProtectedAreasListPanel.tsx
- ├── SpeciesByKingdomPanel.tsx
- ├── rbis/ # RBIS dashboard components
 - ├── ConnectionBar.tsx
 - ├── MetricsPanel.tsx
 - ├── IndicatorsMatrix.tsx
 - ├── SignalFeed.tsx
 - ├── shared/ # RBIS shared UI elements
- ├── ErrorBoundary.tsx
- ├── ProtectedRoute.tsx
- ├── RoleChangeRequestForm.tsx
- ├── RoleChangeApprovalPanel.tsx
- ├── Skeleton.tsx
- ├── hooks/ # Custom React hooks
 - ├── useBiodiversityData.ts
 - ├── useEIAComplianceTracking.ts
 - ├── useGBIF.ts
 - ├── useGBIFOccurrences.ts
 - ├── useLakes.ts
 - ├── useLiveTargetProgress.ts
 - ├── useMapLayers.ts
 - ├── useProtectedAreas.ts
 - ├── useRBIS.ts
 - ├── useRiverNetwork.ts
- ├── pages/ # Page-level components (16 pages)
- ├── services/ # API and business logic services
 - ├── dataService.ts
 - ├── dataValidationService.ts
 - ├── rbisService.ts
 - ├── roleChangeService.ts
 - ├── systemMetricsService.ts
 - ├── automatedProcessingEngine.ts
 - ├── batchUpdateProcessor.ts
 - ├── eventBus.ts
- ├── types/ # TypeScript type definitions
 - ├── index.ts # Core types (User, Indicator, Report, etc.)
 - ├── rbis.ts # RBIS-specific types
 - ├── biodiversity.ts # Biodiversity data types
 - ├── mapLayers.ts # Map layer types
 - ├── overlays.ts # Map overlay types
- ├── utils/ # Utility functions
 - ├── supabase.ts # Supabase client configuration
 - ├── validation.ts # Form validation rules
 - ├── errorHandling.ts # Error handling utilities
 - ├── geoUtils.ts # Geographic calculations
 - ├── hotspotDetection.ts # Biodiversity hotspot algorithms
 - ├── pointClustering.ts # Map point clustering
 - ├── biodiversityCalculations.ts

```
|— threatAssessment.ts
|— rbisFilters.ts      # RBIS search and filter logic
```

5. Interface Specifications

5.1 External API Interfaces

5.1.1 GBIF API (Global Biodiversity Information Facility)

Attribute	Value
Base URL	<code>https://api.gbif.org/v1</code>
Protocol	HTTPS REST
Authentication	None (public API)
Rate Limit	1 request/second (client-side enforced)
Proxy	Supabase Edge Function <code>gbif-proxy</code>

Endpoints Used:

Endpoint	Method	Parameters	Response
<code>/occurrence/search</code>	GET	<code>country=RW</code> , <code>limit</code> , <code>eventDate</code> , <code>hasCoordinate</code>	<code>{ count,</code> <code>results[] }</code>

5.1.2 RBIS API (Rwanda Biodiversity Information System)

Attribute	Value
Base URL	<code>https://rbis.ur.ac.rw/api/v1</code>
Protocol	HTTPS REST
Authentication	API Token (Bearer)
Timeout	10,000ms
Cache Duration	300,000ms (5 minutes)

5.1.3 Supabase API

Attribute	Value
Project URL	<code>https://vivqczyvixdammtaidr.supabase.co</code>
Protocol	HTTPS REST + WebSocket

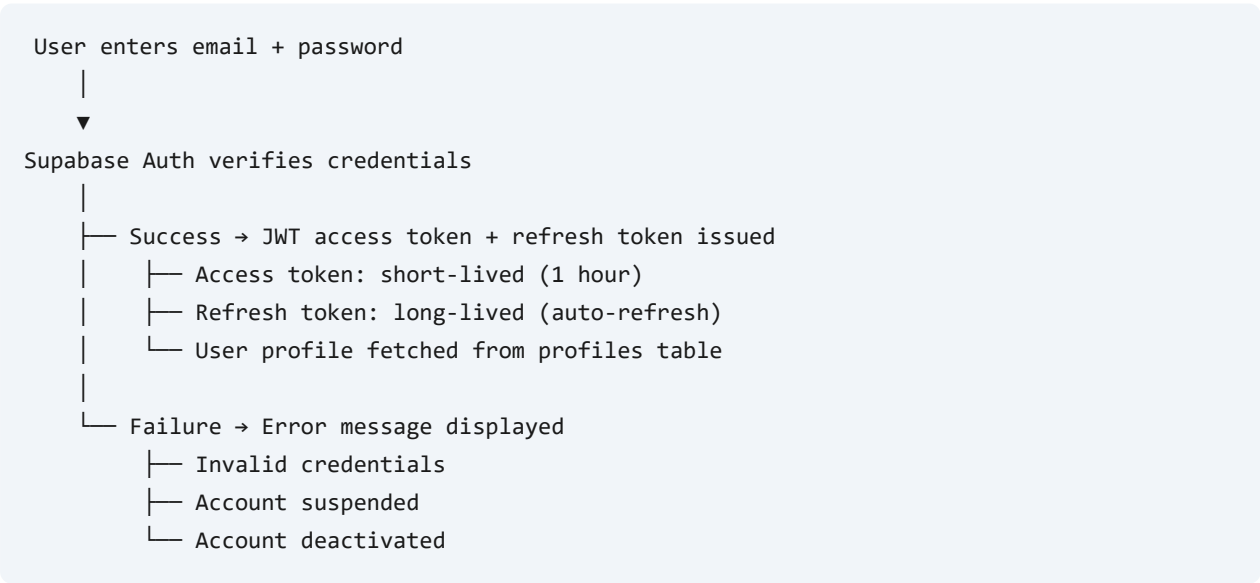
Authentication	JWT (anon key + user session)
RLS	Enabled on all 18 tables

5.2 Internal Service Interfaces

Service	File	Responsibility
<code>dataService.ts</code>	Data CRUD operations	Fetch/create/update reports, indicators, targets
<code>rbisService.ts</code>	RBIS integration	Connection management, metrics, data streams
<code>systemMetricsService.ts</code>	Metrics retrieval	Fetch automated system metrics for dashboard
<code>roleChangeService.ts</code>	Role workflows	Submit/approve/reject role change requests
<code>dataValidationService.ts</code>	Input validation	Validate report form data before submission
<code>automatedProcessingEngine.ts</code>	Batch processing	Automated data processing workflows
<code>eventBus.ts</code>	Event system	In-app event publication and subscription

6. Security Architecture

6.1 Authentication Flow



6.2 Row-Level Security (RLS)

All 18 database tables have RLS enabled with 25+ policies:

Policy Type	Description	Example
SELECT	Users can only read data they are authorised to see	Public viewers see indicators but not audit logs
INSERT	Users can only create data within their scope	District officers insert reports for their district
UPDATE	Users can only modify data they own or are authorised to manage	Admins update user profiles; reporters update own drafts
DELETE	Deletion restricted to authorised roles	Only admins can delete reports

6.3 Security Headers

Header	Value	Purpose
Content-Security-Policy	Restricted script/style sources	Prevent XSS and code injection
X-Frame-Options	DENY	Prevent clickjacking
X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-XSS-Protection	1; mode=block	Browser XSS filter
Strict-Transport-Security	HSTS enabled	Force HTTPS connections

6.4 Data Protection Controls

Control	Implementation
Encryption in transit	TLS 1.2+ on all connections
Encryption at rest	Supabase managed (AES-256)
Password hashing	bcrypt via Supabase Auth
Session management	JWT with automatic refresh
Audit trail	Immutable append-only log table
Input validation	Client-side + database constraints
File upload security	Supabase Storage with access policies

7. Performance Specifications

7.1 Frontend Optimisations

Optimisation	Implementation
Code splitting	Every page lazy-loaded via <code>React.lazy()</code>
Asset caching	Content-hashed filenames with 1-year cache headers
Profile caching	Two-layer cache (memory + sessionStorage) with 5-minute TTL
Stale chunk recovery	Auto-reload when deployment invalidates old chunk URLs
Component memoisation	<code>React.memo</code> on expensive render paths
Debounced search	300ms debounce on search inputs

7.2 Backend Optimisations

Optimisation	Implementation
Database indexes	20+ indexes on frequently filtered columns
JSONB indexing	GIN indexes on <code>form_data</code> for report queries
Connection pooling	Supabase managed connection pooling (PgBouncer)
Edge caching	Static GeoJSON served from CDN edge locations
Real-time push	WebSocket eliminates polling for dashboard updates

7.3 Performance Targets

Metric	Target	Measurement
First Contentful Paint	< 1.5s	Lighthouse audit
Time to Interactive	< 3.0s	Lighthouse audit
API Response (p50)	< 200ms	Supabase dashboard
API Response (p95)	< 500ms	Supabase dashboard
Concurrent Users	500	Load testing
Report Storage	10,000+	Database capacity
Map Render Time	< 2.0s	Chrome DevTools

8. Database Automation

8.1 Trigger Inventory

Trigger	Fires On	Action
---------	----------	--------

on_report_approved	<code>toolkit_reports.status</code> → 'approved'	Updates <code>system_metrics</code> with extracted form values
on_report_deleted	<code>toolkit_reports</code> DELETE (if was approved)	Reverses metric contributions
on_weighted_progress	<code>system_metrics</code> UPDATE	Recalculates target progress using tool weights
on_profile_created	<code>auth.users</code> INSERT	Creates matching row in <code>profiles</code> table
on_role_change_approved	<code>role_change_requests.status</code> → 'approved'	Updates user's role in <code>profiles</code> table
on_notification_created	<code>notifications</code> INSERT	Pushes via Supabase Realtime to connected clients
on_audit_action	Various tables (INSERT/UPDATE/DELETE)	Appends entry to <code>audit_log</code> table

8.2 Database Function Inventory

Function	Purpose
<code>calculate_weighted_progress(target_id)</code>	Computes target progress from tool-weighted metrics
<code>recalculate_all_metrics()</code>	Full recalculation of all 13 system metrics
<code>extract_metric_from_report(report_id)</code>	Extracts metric-relevant values from report JSONB
<code>reverse_metric_contribution(report_id)</code>	Reverses metric impact when an approved report is deleted
<code>get_user_role(user_id)</code>	Returns user's current role from profiles
<code>check_role_permission(user_id, permission)</code>	Checks if user has a specific permission
<code>update_district_compliance(district_id)</code>	Recalculates district compliance score

9. Migration History

Migration	File	Purpose
001	<code>001_initial_schema.sql</code>	Core schema: 15 tables, enums, RLS, indexes
002	<code>002_seed_data.sql</code>	Seed 79 indicators, 22 targets

003	003_add_district_coordinates.sql	Geographic coordinates for 30 districts
004	004_rbis_tables.sql	RBIS integration tables (linkages, data streams, connection log)
005	005_seed_rbis_data_streams.sql	Seed 8 RBIS data streams
006	006_role_change_approval.sql	Role change request workflow
007	007_terminology_update.sql	Terminology standardisation
008	008_add_lakes_table.sql	Lakes geographic data
009	009_add_account_status_fields.sql	Account suspension/deactivation fields
010	010_add_public_viewer_role.sql	Public Viewer role
011	011_rename_sector_reporting_role.sql	Role naming update
012	012_extend_reporting_period.sql	Extended reporting period support
013	013_nbsap_target_integration.sql	Target-report linkage
014	014_update_comprehensive_stakeholder_mapping.sql	Stakeholder mapping update
015	015_comprehensive_reporting_metrics_automation.sql	13 automated metric types
016	016_automatic_reporting_tool_weights.sql	Tool weight configuration
017	017_weighted_progress_triggers.sql	Weighted progress calculation triggers
018	018_ai_extraction_layer.sql	AI extraction tables (subsequently removed)
019	019_delete_cleanup_and_role_cast_fix.sql	Delete cleanup and role type fix
020	020_rwanda_nbsap_2025_2030.sql	Full Rwanda NBSAP 2025–2030 target data and milestones
021	021_target_baselines_and_timelines.sql	Target baselines and timeline data
022	022_drop_ai_extraction_tables.sql	Remove AI extraction tables

10. Testing Strategy

10.1 Test Coverage

Test Type	Scope	Tools
Unit Tests	Service functions, utility functions, hooks	Vitest
Integration Tests	API service → Supabase database round-trips	Vitest + Supabase local
Type Checking	Full codebase type safety	TypeScript compiler (<code>tsc --noEmit</code>)
Linting	Code style and potential errors	ESLint

10.2 Existing Test Files

Test File	Coverage
<code>useEIAComplianceTracking.test.ts</code>	EIA compliance hook logic
<code>useLiveTargetProgress.test.ts</code>	Live target progress calculation
<code>automatedProcessingEngine.test.ts</code>	Automated processing workflows
<code>batchUpdateProcessor.test.ts</code>	Batch update operations
<code>dataValidationService.test.ts</code>	Data validation rules
<code>eventBus.test.ts</code>	Event publication and subscription

11. Deployment Specification

11.1 Environment Configuration

Environment Variable	Required	Description
<code>VITE_SUPABASE_URL</code>	Yes	Supabase project URL
<code>VITE_SUPABASE_ANON_KEY</code>	Yes	Supabase anonymous (public) key
<code>VITE_APP_NAME</code>	No	Application display name
<code>VITE_APP_VERSION</code>	No	Application version string
<code>VITE_APP_ENV</code>	No	Environment identifier (production/staging)
<code>VITE_RBIS_API_BASE_URL</code>	No	RBIS API base URL
<code>VITE_RBIS_API_TOKEN</code>	No	RBIS API authentication token
<code>VITE_RBIS_API_TIMEOUT</code>	No	RBIS API timeout (ms)
<code>VITE_RBIS_CACHE_DURATION</code>	No	RBIS data cache duration (ms)

VITE_ENABLE_RBIS_INTEGRATION	No	Enable/disable RBIS integration
VITE_ENABLE_REALTIME	No	Enable/disable WebSocket real-time updates
VITE_ENABLE_STORAGE	No	Enable/disable file storage

11.2 Build & Deploy Process

```
# Build
npm run build           # Vite production build → dist/

# Output
dist/
├─ index.html           # Entry point
├─ assets/
│   ├─ index-[hash].js  # Main bundle (code-split)
│   ├─ index-[hash].css # Styles
│   └─ [page]-[hash].js # Lazy-loaded page chunks
└─ [static assets]      # GeoJSON, images
```

11.3 Estimated Annual Operating Cost

Service	Tier	Estimated Cost
Vercel	Hobby/Pro	\$0 – \$20/month
Supabase	Free/Pro	\$0 – \$25/month
Domain (if custom)	Standard	\$10 – \$15/year
Total		\$0 – \$555/year

Prepared by: NBSAP Dashboard Development Team Document Reference: NBSAP-SRS-2026-004
 Rwanda Environment Management Authority (REMA)